Bhartiya Vidya Bhavan's, Sardar Patel Institute of Technology
(An Autonomous Institute Affiliated to University of Mumbai)
Bhavan's Campus, Munshi Nagar, Andheri (West),
Mumbai-400058-India

Academic Year: 2025-26 **Class: TE Comp** **Div: D**

Subject: BDA **Sem: 6**

Name	Sneha Ramesh Kawale
UID no.	2024301015
Aim:	Demonstrate use of modern tools like Pandas for Exploratory Data Analysis.

Theory:

1. Introduction

Exploratory Data Analysis (EDA) is an important step in data analysis that helps in understanding the structure, quality, and patterns present in a dataset. It involves summarizing data, detecting missing values, identifying outliers, and studying relationships between variables before applying any machine learning or statistical model.

Modern tools such as **Pandas (Python library)** provide powerful and easy-to-use functions for performing Exploratory Data Analysis efficiently. Pandas allows us to handle large datasets, clean data, and generate useful statistical summaries with very little code.

2. Role of Pandas in Exploratory Data Analysis

Pandas is a high-level data manipulation and analysis library in Python. It provides data structures like **Series** and **DataFrame** which make data handling simple and flexible.

Using Pandas, we can:

- Load datasets from CSV or Excel files.
- View and understand the data structure.
- Detect and handle missing values.
- Remove or correct invalid data.
- Perform statistical analysis.
- Prepare clean data for further processing.

3. Key Operations Performed Using Pandas for EDA

a) Data Loading and Inspection

Pandas is used to read the dataset and examine its basic structure:

- head() and tail() to view sample records.
- shape to check number of rows and columns.
- info() to understand data types and null values.
- describe() to generate statistical summary of numerical columns.

This helps in understanding whether the data is complete and properly formatted.

b) Handling Missing and Invalid Values

Real-world datasets often contain missing or incorrect values such as null entries or negative values where they are not meaningful.

Using Pandas, we can:

- Detect missing values using isnull() and sum().
- Replace missing values with mean, median, or mode depending on the data type.
- Correct invalid values (such as negative prices) by converting them to valid statistical values.

Example:

- Numerical data → filled using **median**
- Categorical data → filled using **mode**

c) Data Type Conversion

Pandas allows conversion of data types for proper analysis:

- Converting date columns using to_datetime()
- Converting numerical values stored as strings into numeric format

This ensures that calculations and visualizations are accurate.

d) Outlier Detection

Outliers are extreme values that differ significantly from other observations and can affect analysis results.

Using Pandas, outliers can be detected using:

- Quartiles (Q1, Q3)
- Interquartile Range (IQR)

Records lying outside the acceptable range are identified and can be removed or treated.

e) Data Distribution Analysis

EDA also includes understanding how data is distributed:

- Checking if data is normal or skewed
- Identifying patterns and trends

Histograms and summary statistics help in visualizing these distributions.

Python Code:

```
import pandas as pd
import numpy as np
```

```
data = pd.read_excel("BDA_sheet.xlsx")
data.head(5)
```

	Transaction ID	Customer ID	category	Item	Price Per Unit	Quantity	Total Spent	Discount given	Payment Method	Transaction Id	Card type	Location	Pincode	Date
0	TXN_6867343	CUST_09	Home	Item_01	5.0	1.0	-2.0	7.0	Cash	NaN	NaN	Bangur Nagar	400090.0	NaN
1	TXN_7482416	CUST_09	Grocery	Item_10_PAT	18.5	8.0	136.0	12.0	Credit Card	738196052.0	Visa	Kurl	400070.0	2024-04-08 00:00:00
2	TXN_4413070	CUST_14	Home	Item_01	5.0	8.0	31.0	9.0	Credit Card	746724054.0	Visa	Parel	400012.0	11/31/2023
3	TXN_7563311	CUST_23	Grocery	Item_10_PAT	18.5	2.0	27.0	10.0	Google Pay	532940866.0	NaN	Kurla	400070.0	2022-05-20 00:00:00
4	TXN_7138501	CUST_18	Home	Item_01	5.0	1.0	-10.0	15.0	Credit Card	628129987.0	Visa	Sewree	400015.0	2024-11-16 00:00:00

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3069 entries, 0 to 3068
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Transaction ID   3068 non-null   object
 1   Customer ID      3068 non-null   object
 2   category         3068 non-null   object
 3   Item             2841 non-null   object
 4   Price Per Unit   2948 non-null   float64
 5   Quantity         3068 non-null   float64
 6   Total Spent      3068 non-null   float64
 7   Discount given   3068 non-null   float64
 8   Payment Method   3068 non-null   object
 9   Transaction Id   1633 non-null   float64
 10  Card type        1044 non-null   object
 11  Location         3068 non-null   object
 12  Pincode          3068 non-null   float64
 13  Date             3068 non-null   object
dtypes: float64(6), object(8)
memory usage: 335.8+ KB
```

```
# now remove the 2nd transaction id column
data.drop(data.columns[9], axis=1, inplace=True)
data.head(5)
```

	Transaction ID	Customer ID	category	Item	Price Per Unit	Quantity	Total Spent	Discount given	Payment Method	Card type	Location	Pincode	Date
0	TXN_6867343	CUST_09	Home	Item_01	5.0	1.0	-2.0	7.0	Cash	NaN	Bangur Nagar	400090.0	NaN
1	TXN_7482416	CUST_09	Grocery	Item_10_PAT	18.5	8.0	136.0	12.0	Credit Card	Visa	Kurl	400070.0	2024-04-08 00:00:00
2	TXN_4413070	CUST_14	Home	Item_01	5.0	8.0	31.0	9.0	Credit Card	Visa	Parel	400012.0	11/31/2023
3	TXN_7563311	CUST_23	Grocery	Item_10_PAT	18.5	2.0	27.0	10.0	Google Pay	NaN	Kurla	400070.0	2022-05-20 00:00:00
4	TXN_7138501	CUST_18	Home	Item_01	5.0	1.0	-10.0	15.0	Credit Card	Visa	Sewree	400015.0	2024-11-16 00:00:00

```
data.isnull().sum()
```

	0
Transaction ID	1
Customer ID	1
category	1
Item	228
Price Per Unit	121
Quantity	1
Total Spent	1
Discount given	1
Payment Method	1
Card type	2025
Location	1
Pincode	1
Date	1

dtype: int64

```
#remove the null transaction id as it's only one

data = data[data["Transaction ID"].notna()]
data.isnull().sum()
```

	0
Transaction ID	0
Customer ID	0
category	0
Item	227
Price Per Unit	120
Quantity	0
Total Spent	0
Discount given	0
Payment Method	0
Card type	2024
Location	0
Pincode	0
Date	1

dtype: int64

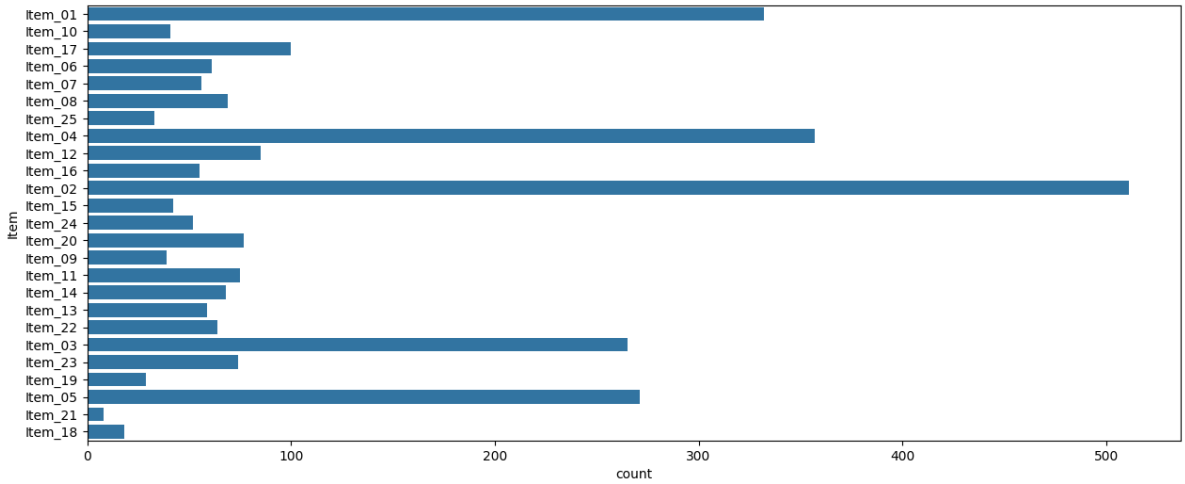
```
print("Item unique values ",data["Item"].unique())

data["Item"] = data["Item"].str.lower()
data["Item"] = data["Item"].str.replace(r"[.,]", "", regex=True)
data["Item"] = data["Item"].str.replace("_pat", "", regex=False)
data["Item"] = data["Item"].str.extract(r"(\d+)")
data["Item"] = "Item_" + data["Item"].str.zfill(2)
```

```
print("Item unique values after processing",data["Item"].unique())
```

```
Item unique values  ['Item_01' 'Item_10_PAT' 'Item_17_PAT' 'it,06' 'it,07' 'it,08'
 'Item_25_PAT' 'Item_4_PAT' 'Item_12_PAT' 'Item_6_PAT' 'Item_16_PAT'
 'Item_02' 'Item_7_PAT' 'Item_15_PAT' 'Item_24_PAT' nan 'Item_20_PAT'
 'Item_9_PAT' 'Item_8_PAT' 'Item_11_PAT' 'Item_14_PAT' 'Item_2_PAT'
 'Item_13_PAT' 'Item_22_PAT' 'Item_1_PAT' 'item_03' 'Item_23_PAT'
 'Item_19_PAT' 'item_04' 'Item_5_PAT' 'Item_3_PAT' 'item_05' 'ite,m_01'
 'Item_21_PAT' 'Item.03' 'ite,m.05' 'item_01' 'it01' 'it03' 'it04' 'it02'
 'Item_18_PAT' 'item01' 'item_02' 'it,o3' 'it_oo5' 'item02' 'it,m_02'
 'ite,05' 'ite,03' 'item,01' 'i01' 'io2' 'io1' 'io4' 'io5' 'itm_17'
 'item,o4' 'ite,m02' 'item04' 'item,m01' 'ie03' 'ie05' 'itm,03' 'item_1'
 'itm_02' 'ite,m01' 'item,m02']
Item unique values after processing ['Item_01' 'Item_10' 'Item_17' 'Item_06' 'Item_07' 'Item_08' 'Item_25'
 'Item_04' 'Item_12' 'Item_16' 'Item_02' 'Item_15' 'Item_24' nan 'Item_20'
 'Item_09' 'Item_11' 'Item_14' 'Item_13' 'Item_22' 'Item_03' 'Item_23'
 'Item_19' 'Item_05' 'Item_21' 'Item_18']
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
plt.figure(figsize=(15,6))
sns.countplot(data["Item"])
plt.show()
```

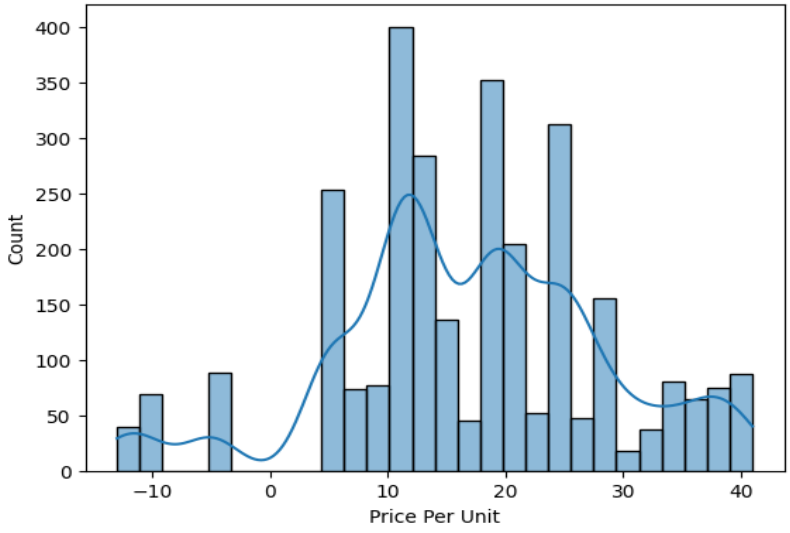



```
data["Item"] = data["Item"].fillna(data["Item"].mode()[0])
data["Item"].isnull().sum()
```

```
np.int64(0)
```

```
sns.histplot(data["Price Per Unit"],kde=True)
```

```
<Axes: xlabel='Price Per Unit', ylabel='Count'>
```




```
print(data["Price Per Unit"].mean())
data["Price Per Unit"] = data["Price Per Unit"].fillna(data["Price Per Unit"].mean())
data["Price Per Unit"].isnull().sum()
```

```
17.046811397557665
np.int64(0)
```

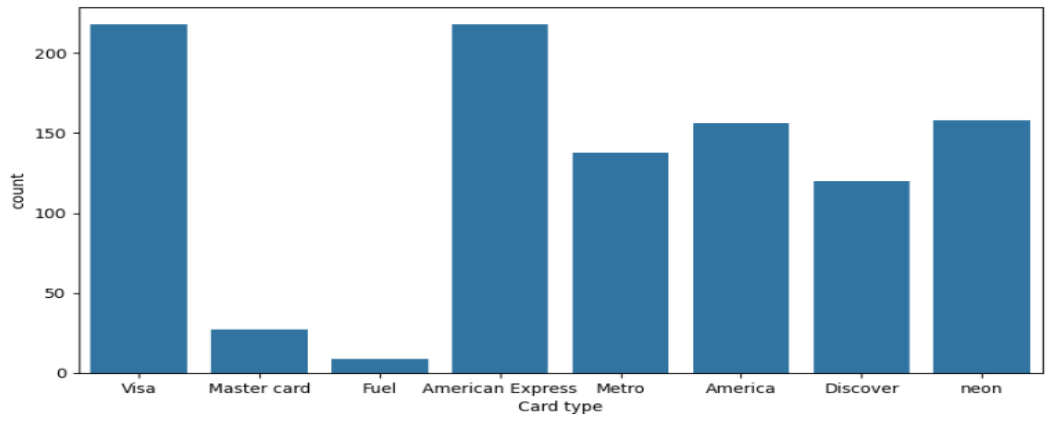
```
data["Card type"].unique()
```

```
array([nan, 'Visa', 'Master card', 'Fuel', 'American Express', 'Vis',
       'Mastero', 'A Exp', 'America', 'Discover', 'Viza', 'neon', 'VISA',
       'MaTERO', 'Amareica'], dtype=object)
```

```
data["Card type"] = data['Card type'].replace(["VISA","Vis","Viza"],"Visa")
data["Card type"] = data['Card type'].replace("A Exp","American Express")
data["Card type"] = data['Card type'].replace(["Mastero","MaTERO"],"Metro")
data["Card type"] = data['Card type'].replace("Amareica","America")

mode_value = data["Card type"].mode()[0]
print(mode_value)

data["Card type"] = data["Card type"].replace("neon",mode_value)

data["Card type"].unique()
```

```
American Express
array(['American Express', 'Visa', 'Master card', 'Fuel', 'Metro',
       'America', 'Discover'], dtype=object)
```

```
plt.figure(figsize=(10,5))
sns.countplot(x='Card type', data=data)
plt.show()
```




```
print(data["Card type"].mode()[0])
data["Card type"] = data["Card type"].fillna(data["Card type"].mode()[0])
data["Card type"].isnull().sum()
```

```
American Express
np.int64(0)
```

```
data["Location"].unique()
```

```
array(['Bangur Nagar', 'Kurl', 'Parel', 'Kurla', 'Sewree', 'Antop Hill',
       'Grantt Road', 'Nehru Nagar', 'Malad', 'Dharavi', 'Zumbala Hill',
       'Trombay', 'Kalbadevi', 'Antope', 'Grat Road', 'Malbar Hill',
       'Mumbai Central', 'Bangurr', 'Mandvi', 'Antop ', 'Siwree',
       'Kalba devi', 'Manad', 'Mandvir', 'Cumbala Hills', 'Parle',
       'Malda', 'N Nagar', 'Cum Hills', 'Tilak', 'antipe Hill',
       'Bang Nagar', 'Sewri', 'Gratt Road', 'Ant Hill', 'Cumba Hills',
       'Paral', 'Mandiv', 'karla', 'Bangur Nagr', 'hutam hills',
       'ala Hills', 'Jogeshwari', 'Kaldadevi', 'ant hil;', 'Bagur Nagar',
       'kurle', 'Bangur Naar', 'perul', 'ant Hills', 'Mandivs',
       'gat road', 'Bangr Nagar', 'any hill', 'Sewre', 'Sion', 'bandra',
       'Ghatkopar', 'Nenru Nagar', 'Kurda', 'Parul', 'sewri', 'Gat',
       'Kalda devi', 'Colaba', 'Powai', 'colar', 'Puwai', 'Tilak Nagar',
       'Tilak Nagare', 'mandvi', 'seepz', 'Pouai', 'NITIE', 'mandir',
       'Nehru Nagare', 'gran road', 'Cion', 'Anderi', 'Kaibadevi',
       'T Nagar', 'nite', 'nahur', 'Hutatma Chowk', 'zeepz', 'Seepz',
       'malad', 'Powae', 'Mandar', 'Seon', 'Santacruz', 'Melad',
       'mazgoan', 'Kappadevi', 'Tank Road', 'colala', 'Aarey Milk colony',
       'Tilk Nagar', 'Malade', 'Aarey ', 'Colala', 'sewre', 'Powair',
       'H Chowk', 'Aarey Milk', 'Anand Parbat', 'seep', 'Ghacopar',
       'coala', 'any Hill', 'kandivili', 'kuda', ' Milk colony', 'hill',
       'mulund', 'Hutma Chowk', 'Bandra(W)', 'Mazgoan', 'joghwari',
       'parel', 'powau', 'Aare', 'Ghatcopar', 'Aarey colony', 'zantel',
       'jogeshwar', 'ghacopar', 'antop', 'Hut Chowk', 'Ghatkopr', 'tromb',
       'andri', 'naintal', 'Tilak ', 'Ghatkopir', 'sion', 'nitie',
       'kurla', 'kalbha devi', 'poawi', 'sewree', 'dharavi'], dtype=object)
```

```
!pip install rapidfuzz
```

```
Requirement already satisfied: rapidfuzz in /usr/local/lib/python3.12/dist-packages (3.14.3)
```

```
correct_locations = [
    "Kurla","Grant Road","Sewree","Malabar Hill","Bangur Nagar","Mandvi",
    "Kalbadevi","Powai","Nehru Nagar","Ghatkopar","Andheri","Jogeshwari",
    "Aarey Colony","Tilak Nagar","Hutatma Chowk","Bandra","Colaba",
    "Parel","Malad","Sion","Dharavi","Santacruz","Mulund","Mazgaon"
]

from rapidfuzz import process

def correct_location(name):
    match, score, _ = process.extractOne(name, correct_locations)
    return match if score > 70 else name   # threshold

data["Location"] = data["Location"].apply(correct_location)

data["Location"].unique()
```

```
array(['Bangur Nagar', 'Kurla', 'Parel', 'Sewree', 'Antop Hill',
       'Grant Road', 'Nehru Nagar', 'Malad', 'Dharavi', 'Zumbala Hill',
       'Trombay', 'Kalbadevi', 'Antope', 'Malabar Hill', 'Mumbai Central',
       'Mandvi', 'Antop ', 'Cumbala Hills', 'Cum Hills', 'Tilak Nagar',
       'antipe Hill', 'Cumba Hills', 'karla', 'hutam hills', 'Jogeshwari',
       'ant hil;', 'kurle', 'perul', 'ant Hills', 'gat road', 'any hill',
       'Sion', 'Bandra', 'Ghatkopar', 'sewri', 'Gat', 'Colaba', 'Powai',
       'colar', 'seepz', 'NITIE', 'mandir', 'Andheri', 'nite', 'nahur',
       'Hutatma Chowk', 'zeepz', 'Seepz', 'Mandar', 'Santacruz',
       'Mazgaon', 'colala', 'Aarey Colony', 'Aarey Milk', 'Anand Parbat',
       'seep', 'coala', 'kuda', ' Milk colony', 'Mulund', 'powau',
       'zantel', 'tromb', 'naintal', 'nitie', 'poawi'], dtype=object)
```

```
data.isnull().sum()
data["Pincode"] = data["Pincode"].astype(int)
data["Pincode"].unique()

#there are many correct pincode but there are many wrong pincode with the wrong leng
```

```
array([ 400090,  400070,  400012,  400015,  410037,  400007,  400024,
        400097,  400017,  400026,  400073,  400002,  400064,  400006,
        400008,  403070,  401097,  400003,  400035, 4010064, 4000614,
        400126,  400095, 4000208, 4000276, 4001006,  420070,  400014,
        410002, 4000947,  410064,  400197, 4000006,  400190,  410045,
```

```
 4020202,  401008,  400087,  403069,  400020,  410036,  400091,
  400043,  405043,  400033,  402069,  400065,  400465, 4020022,
  402097,  400023,  400077,  440043,  400062,  401077,  400049,
  401036,  110005,  400093, 1400077,  400569,  400022, 4000165,
   41004,  400010,   40043,  401087,  410040,  401096,  400096,
  400136,  400066,  400177,  410069,  400196,  120005,   40073,
  400055,  400336,  400086,  403036, 1120005,  110006,  410043,
 4000565,  400009,  400063,  403043, 4010036, 4000136,  400443,
 4000890, 4000809,  400031,  410089, 4003043, 4000323,  400079,
 4000069, 4000336,  400343, 1170005,  400099, 4010096,  401069,
  401065,   40004, 4005023,   40003])
```

```
print(data[["Location","Pincode"]])

data["Pincode"] = data["Pincode"].astype(str)
data.loc[data["Pincode"].str.len() != 6, "Pincode"] = None

data["Pincode"] = data.groupby("Location")["Pincode"].transform(
    lambda x: x.fillna(x.mode()[0] if not x.mode().empty else None)
)

mode_pin = data["Pincode"].mode()[0]
data["Pincode"] = data["Pincode"].fillna(mode_pin)

data["Pincode"].str.len().value_counts()
```

```
           Location  Pincode
0     Bangur Nagar   400090
1            Kurla   400070
2            Parel   400012
3            Kurla   400070
4           Sewree   400015
...            ...      ...
3063     Santacruz   400043
3064       Andheri   400069
3065         NITIE   400087
3066    Grant Road   400033
3067       Andheri   400069

[3068 rows x 2 columns]
```

	count

Pincode	
6	3068

dtype: int64

```
data.isnull().sum()
```

	0
Transaction ID	0
Customer ID	0
category	0
Item	0
Price Per Unit	0
Quantity	0
Total Spent	0
Discount given	0
Payment Method	0
Card type	0
Location	0
Pincode	0
Date	1

dtype: int64

```
data["Date"]
```

	Date
0	NaN
1	2024-04-08 00:00:00
2	11/31/2023
3	2022-05-20 00:00:00
4	2024-11-16 00:00:00
...	...
3063	2024-05-13 00:00:00
3064	2022-08-18 00:00:00
3065	2024-08-20 00:00:00
3066	2024-11-28 00:00:00
3067	2024-05-01 00:00:00

3068 rows × 1 columns

dtype: object

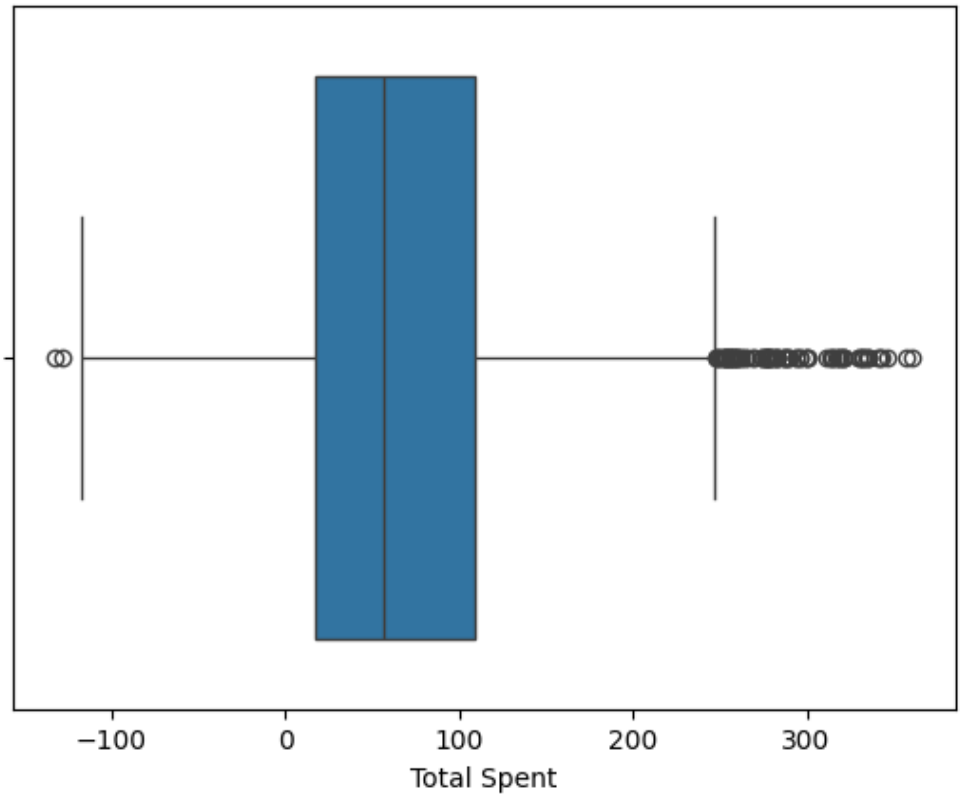
```
data["Date"] = pd.to_datetime(data["Date"], errors="coerce")
print(data["Date"])
print(data["Date"].isnull().sum())

data["Date"] = data["Date"].fillna(data["Date"].mode()[0])
print("After filling the na values ",data["Date"].isnull().sum())
```

```
0            NaT
1     2024-04-08
2            NaT
3     2022-05-20
4     2024-11-16
         ...
3063  2024-05-13
```

```
3063   2024-05-13
3064   2022-08-18
3065   2024-08-20
3066   2024-11-28
3067   2024-05-01
Name: Date, Length: 3068, dtype: datetime64[ns]
13
After filling the na values  0
```

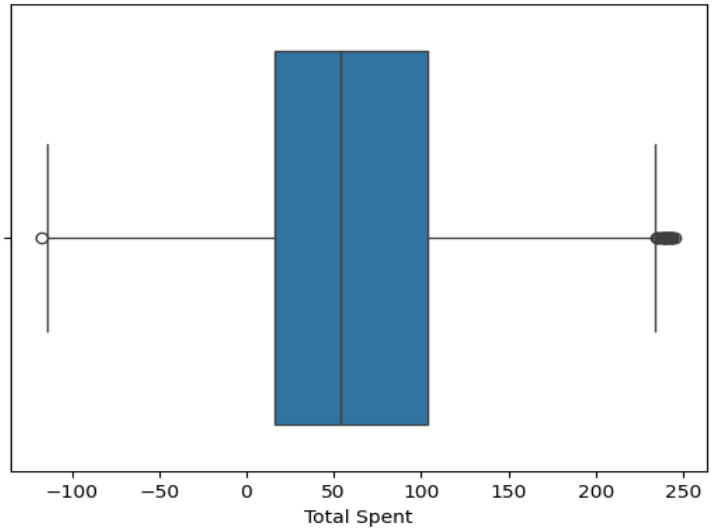
```
sns.boxplot(data=data,x="Total Spent")
data.shape
```

```
(3068, 13)
```



```
Q1 = data["Total Spent"].quantile(0.25)
Q3 = data["Total Spent"].quantile(0.75)
IQR = Q3 - Q1

left_side_outliers = Q1 - 1.5 * IQR
right_side_outliers = Q3 + 1.5 * IQR

data = data[(data["Total Spent"] > left_side_outliers) & (data["Total Spent"] < right_side_outliers)]
sns.boxplot(data=data,x="Total Spent")
```

```
<Axes: xlabel='Total Spent'>
```




```
data.shape
```

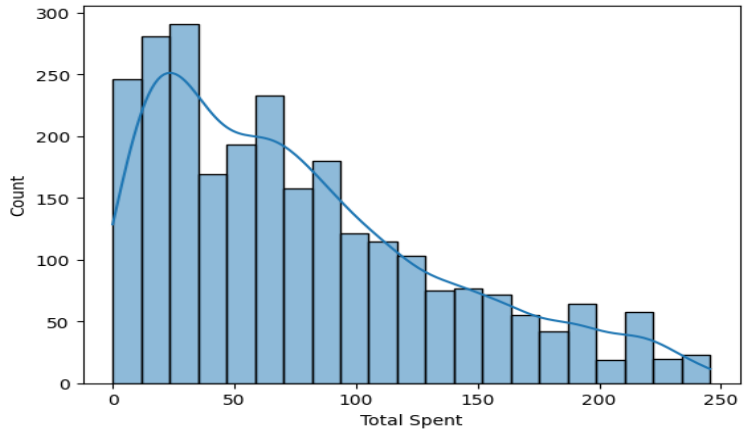
```
(2978, 13)
```

```
data.loc[data["Total Spent"] < 0 , "Total Spent"] = np.nan
data["Total Spent"].isnull().sum()
```

```
np.int64(383)
```

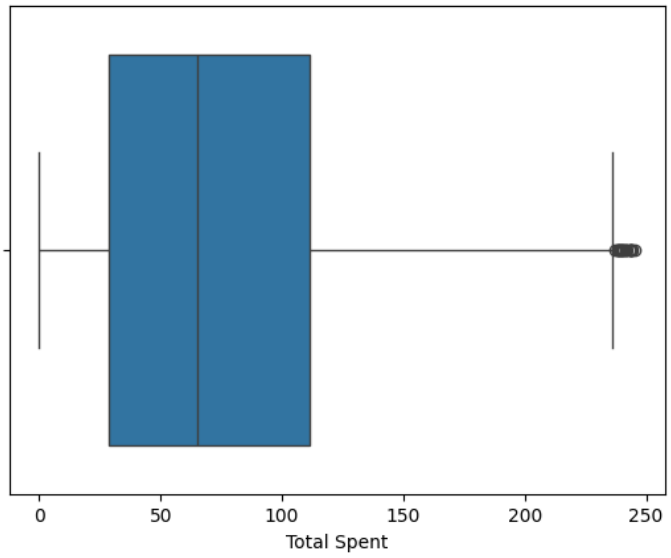
```
sns.histplot(data["Total Spent"],kde=True)
```

```
<Axes: xlabel='Total Spent', ylabel='Count'>
```



```
sns.boxplot(data=data,x="Total Spent")
#as we have normaly data right skewed removing outliers will not affect that much for normal distribution
```

```
<Axes: xlabel='Total Spent'>
```




```
data["Total Spent"] = data["Total Spent"].fillna(data["Total Spent"].median())
```

```
data[data["Price Per Unit"] < 0 ].count()
```

	0
Transaction ID	195
Customer ID	195
category	195
Item	195
Price Per Unit	195
Quantity	195
Total Spent	195
Discount given	195
Payment Method	195
Card type	195
Location	195
Pincode	195
Date	195

dtype: int64

```
print(data["Price Per Unit"].unique())

data.loc[data["Price Per Unit"] < 0, "Price Per Unit"] = np.nan

data["Price Per Unit"] = data["Price Per Unit"].fillna(data["Price Per Unit"].median())

data["Price Per Unit"].unique()
```

```
data["Price Per Unit"].unique()

print(data.groupby("Item")["Price Per Unit"].unique())

data["Price Per Unit"] = data.groupby("Item")["Price Per Unit"].transform(
    lambda x: x.mode()[0]
)

print(data.groupby("Item")["Price Per Unit"].unique())
```

```
[ 5.  18.5 29.  12.5 14.  15.5 19.  21.5 27.5 11.  26.  39.5 33.5 17.
 41.  20.  24.5 23.  36.5 13.  38.  32.  25.  35.  30.5]
Item
Item_01     [5.0]
Item_02    [11.0]
Item_03    [13.0]
Item_04    [19.0]
Item_05    [25.0]
Item_06    [12.5]
Item_07    [14.0]
Item_08    [15.5]
Item_09    [17.0]
Item_10    [18.5]
Item_11    [20.0]
Item_12    [21.5]
Item_13    [23.0]
Item_14    [24.5]
Item_15    [26.0]
Item_16    [27.5]
Item_17    [29.0]
Item_18    [30.5]
Item_19    [32.0]
Item_20    [33.5]
Item_21    [35.0]
Item_22    [36.5]
Item_23    [38.0]
Item_24    [39.5]
Item_25    [41.0]
Name: Price Per Unit, dtype: object
Item
Item_01     [5.0]
Item_02    [11.0]
Item_03    [13.0]
Item_04    [19.0]
Item_05    [25.0]
Item_06    [12.5]
Item_07    [14.0]
Item_08    [15.5]
Item_09    [17.0]
Item_10    [18.5]
```

```
Item_11    [20.0]
Item_12    [21.5]
Item_13    [23.0]
Item_14    [24.5]
Item_15    [26.0]
Item_16    [27.5]
Item_17    [29.0]
Item_18    [30.5]
Item_19    [32.0]
Item_20    [33.5]
Item_21    [35.0]
Item_22    [36.5]
Item_23    [38.0]
Item_24    [39.5]
Item_25    [41.0]
Name: Price Per Unit, dtype: object
```

```
print(data["Payment Method"].unique())

data["Payment Method"] = data["Payment Method"].replace(["Gpay","Google","Google "],"Google Pay")
data["Payment Method"] = data["Payment Method"].replace(["Ptm","Patm"],"Paytm")
data["Payment Method"] = data["Payment Method"].replace(["Bhima","Bheema "],"Bhim Pay")
data["Payment Method"] = data["Payment Method"].replace(["Amzon"],"Amazon Pay")
data["Payment Method"] = data["Payment Method"].replace(["Rupay","Rpay"],"Rupee pay")
data["Payment Method"] = data["Payment Method"].replace(["epay"],"Epay")

print(data["Payment Method"].unique())
```

```
['Cash' 'Credit Card' 'Google Pay' 'Rupee pay' 'Bhim Pay' 'Paytm'
 'Amazon Pay' 'Epay']
['Cash' 'Credit Card' 'Google Pay' 'Rupee pay' 'Bhim Pay' 'Paytm'
 'Amazon Pay' 'Epay']
```

```
data["Year"] = data["Date"].dt.year
data["Month"] = data["Date"].dt.month
data["MonthDate"] = data["Date"].dt.date
```

```
data.head(10)
```

	Transaction ID	Customer ID	category	Item	Price Per Unit	Quantity	Total Spent	Discount given	Payment Method	Card type	Location	Pincode	Date	Year	Month	MonthDate
0	TXN_6867343	CUST_09	Home	Item_01	5.0	1.0	65.0	7.0	Cash	American Express	Bangur Nagar	400090	2022-08-13	2022	8	2022-08-13
1	TXN_7482416	CUST_09	Grocery	Item_10	18.5	8.0	136.0	12.0	Credit Card	Visa	Kurla	400070	2024-04-08	2024	4	2024-04-08
2	TXN_4413070	CUST_14	Home	Item_01	5.0	8.0	31.0	9.0	Credit Card	Visa	Parel	400012	2022-08-13	2022	8	2022-08-13
3	TXN_7563311	CUST_23	Grocery	Item_10	18.5	2.0	27.0	10.0	Google Pay	American Express	Kurla	400070	2022-05-20	2022	5	2022-05-20
4	TXN_7138501	CUST_18	Home	Item_01	5.0	1.0	65.0	15.0	Credit Card	Visa	Sewree	400015	2024-11-16	2024	11	2024-11-16
5	TXN_4353295	CUST_22	Home	Item_01	5.0	4.0	8.0	12.0	Google Pay	American Express	Antop Hill	410037	2022-03-10	2022	3	2022-03-10
6	TXN_1621497	CUST_06	Grocery	Item_17	29.0	7.0	192.0	11.0	Credit Card	Visa	Grant Road	400007	2022-04-05	2022	4	2022-04-05
7	TXN_9331642	CUST_12	Home	Item_01	5.0	6.0	17.0	13.0	Cash	American Express	Nehru Nagar	400024	2023-02-18	2023	2	2023-02-18
8	TXN_1132097	CUST_24	Home	Item_01	5.0	7.0	29.0	6.0	Credit Card	Visa	Malad	400097	2022-07-06	2022	7	2022-07-06
9	TXN_5885894	CUST_01	Grocery	Item_06	12.5	1.0	11.0	12.0	Cash	American Express	Kurla	400070	2023-05-18	2023	5	2023-05-18

conclusion	In this experiment, Pandas was successfully used as a modern tool for Exploratory Data Analysis (EDA). The dataset was analyzed to understand its structure, handle missing and invalid values, and detect outliers. Statistical summaries and visualizations helped in identifying patterns and data distribution. This experiment showed that Pandas makes data analysis easier, faster, and more accurate. Hence, Exploratory Data Analysis is an essential step for preparing clean and reliable data for further processing and decision-making.